

Sprint 1: System Architecture & Scope Definition

Section 1: Target Audience & Market Focus

Primary Persona: Ahmed, 20–30 years old, a style-conscious retail consumer who shops for clothing primarily on his phone, usually in short sessions between classes or work. He compares sizes and prices across a few items before buying and abandons purchases when sizing information is unclear or the checkout process takes too many steps.

Core Pain Point: Small apparel retailers' existing sites frequently show items as available that are actually out of stock in a given size, forcing post-payment order cancellations, while search/filter options are usually too shallow (no size, color, or price-range filtering) to let users narrow down a catalog efficiently.

Domain Scope: Apparel (men's and women's casual clothing — shirts, jeans, jackets) — chosen because product variation is bounded to a manageable set of attributes (size, color, category), inventory can be modeled per size/variant without needing a highly complex configurator, and the domain is realistic to fully implement end-to-end within a single academic semester.

Section 2: MVP Feature Scope Matrix

Category	Feature Name	Description	Priority
Authentication	User Registration & Authentication	Password hashing (bcrypt) and JWT-based authentication for signup/login.	High (MVP)
Catalog	Product List & Search	Product browsing interface with category, size, and price-range filtering.	High (MVP)
Cart	Cart Management	State-persistent cart management (item addition, size/quantity modification, deletion), tied to the user's account.	High (MVP)
Checkout	Order Processing	Mock/Stripe test-mode payment gateway integration and order object instantiation with status tracking.	High (MVP)
Admin	Inventory Control	Administrative CRUD operations for product and per-size stock-quantity management.	Medium
Account	Order History	Authenticated users can view past orders and their current status.	Medium

Feasibility note: The four High (MVP) features form the minimum functional loop (browse → cart → pay → order created) needed for the app to be usable end-to-end. The two Medium features extend usability without expanding into out-of-scope territory (e.g., returns/exchanges, wishlists, multi-vendor support) that would be unrealistic for a solo build in one semester.

Section 3: Tech Stack Selection & Justification

- **Frontend Framework: React (with Vite)** Justification: React's component-based structure maps cleanly onto the catalog/cart/checkout split, and its large ecosystem (React Router, Context/Redux for cart state) is well-documented for a solo developer under a deadline — a lighter setup path than a full Next.js SSR project, which offers SEO/server-rendering benefits this project doesn't need.
- **Backend Infrastructure: Node.js / Express** Justification: Express keeps the API layer minimal and lets JavaScript be used across both frontend and backend, reducing context-switching; its middleware pattern also fits JWT auth and request validation without the added structural overhead of a framework like Django or Spring Boot, whose scalability features exceed this project's scale.

- **Database Management System: PostgreSQL** Justification: The domain (users, products with size/variant stock, orders, order items) has strict referential integrity needs — an order must reference a valid product and size variant with accurate quantities — which PostgreSQL's foreign-key constraints and transactional guarantees handle well, versus a document store like MongoDB, which would push that integrity logic into the application layer.
 - **Caching & Asynchronous Processing (Optional): Redis** Justification: Redis caches catalog reads (reducing repeated DB load on browse/search endpoints) and stores active cart session data for fast reads/writes while a user is shopping.
-

Section 4: Entity-Relationship Diagram (ERD)

Cardinality & Relationships

- **Users → Orders: 1:N** (one user places many orders)
- **Users → Cart: 1:1** (one active cart per user)
- **Cart → Cart_Items: 1:N**
- **Products → Cart_Items: 1:N**
- **Orders → Order_Items: 1:N**
- **Products → Order_Items: 1:N**
- **Categories → Products: 1:N**

Attribute Definitions & Keys

Entity	Attribute	Type	Key
Users	id	INTEGER	PK
Users	email	VARCHAR(255)	—
Users	password_hash	VARCHAR(255)	—
Users	created_at	TIMESTAMP	—
Categories	id	INTEGER	PK
Categories	name	VARCHAR(100)	—
Products	id	INTEGER	PK
Products	category_id	INTEGER	FK → Categories.id
Products	name	VARCHAR(150)	—
Products	size	VARCHAR(10)	—
Products	color	VARCHAR(50)	—
Products	price	DECIMAL(10,2)	—
Products	stock_quantity	INTEGER	—
Cart	id	INTEGER	PK
Cart	user_id	INTEGER	FK → Users.id
Cart_Items	id	INTEGER	PK
Cart_Items	cart_id	INTEGER	FK → Cart.id
Cart_Items	product_id	INTEGER	FK → Products.id
Cart_Items	quantity	INTEGER	—
Orders	id	INTEGER	PK
Orders	user_id	INTEGER	FK → Users.id

Entity	Attribute	Type	Key
Orders	total_amount	DECIMAL(10,2)	—
Orders	status	VARCHAR(50)	—
Orders	created_at	TIMESTAMP	—
Order_Items	id	INTEGER	PK
Order_Items	order_id	INTEGER	FK → Orders.id
Order_Items	product_id	INTEGER	FK → Products.id
Order_Items	quantity	INTEGER	—
Order_Items	unit_price	DECIMAL(10,2)	—

Mermaid ERD

